

• Sec-A

1. Explain the concept of amortized analysis and how it is used to analyze the time and space complexity of algorithm with varying cost for individual operations.

Ans Amortized analysis is a technique used to analyze the average-case running time of a sequence of operations. It is particularly useful when individual operations can have widely varying costs, but the average cost over a sequence of operations is significantly lower.

2. The minimum no. of comparisons required to find the minimum and the maximum of 100 numbers is:-

Ans Using Divide and Conquer to achieve minimum comparison:

$$n = 100$$

$$\text{Minimum no. of comparisons} = \frac{3n}{2} - 2 = 3\left(\frac{100}{2}\right) - 2 = 148$$

3. Provide example of real-world problems or applications where divide and conquer algorithms are commonly used.

Ans

- (i) Merge Sort
- (ii) Median Finding
- (iii) Min and max finding
- (iv) Matrix Manipulation
- (v) Closest pair problem

4. Define the Divide and Conquer algorithm paradigm in Computer science.

Ans In Computer science, divide and conquer is an algorithm design paradigm. A divide and conquer algorithm recursively breaks down a problem into two or more sub-problem of the same or related type, until they become simple enough to be solved directly.

5. Explain the three main steps involved in a Divide and Conquer algorithm.

Ans The three main steps involved in a Divide and Conquer algorithm:

1: Divide the original problem into a set of subproblems

2: Conquer: Solve every subproblem individually, recursively.

3: Combine: Put together the solutions of the subproblems to get the solution to the whole problem.

Section B

1. Iteration method

$$(i) T(n) = T(n-1) + 1/n$$

$$(ii) T(n) = T(n/3) + n^{1/3}$$

Solⁿ (i) $T(n) = T(n-1) + 1/n$ — ①

$$T(n-1) = T(n-2) + \frac{1}{n-1}$$

$$T(n-2) = T(n-3) + \frac{1}{n-2}$$

Put $T(n-1)$ in eqⁿ ①

$$T(n) = T(n-2) + \frac{1}{n-1} + \frac{1}{n}$$

Put $T(n-2)$ in above eqⁿ

$$T(n) = T(n-3) + \frac{1}{n-2} + \frac{1}{n-1} + \frac{1}{n}$$

$$T(n) = T(1) + \sum_{i=1}^n \frac{1}{i}$$

(ii) $T(n) = T(n/3) + n^{4/3}$
 $T(n/3) = T(n/9) + (n/9)^{4/3}$ — (1)
 Put $T(n/3)$ in eqn (1)

$$T(n) = T(n/9) + (n/9)^{4/3} + n^{4/3} \quad \text{--- (2)}$$

$$T(n/9) = T(n/27) + (n/9)^{4/3}$$

Put $T(n/9)$ in eqn (2)

$$T(n) = T(n/27) + (n/9)^{4/3} + (n/9)^{4/3} + n^{4/3}$$

We can generate

$$T(n) = T(n/3^k) + \sum_{j=0}^{k-1} (n/3^j)^{4/3}$$

Calculate the sum :- for k such that $n/3^k = 1$

$$\frac{n}{3^k} = 1$$

$$n = 3^k$$

$$k = \log_3 n$$

$$T(n) = T(1) + n^{4/3} \sum_{j=0}^{\log_3 n - 1} \frac{1}{3^{4j/3}}$$

The sum is a GP $\sum_{j=0}^{\log_3 n - 1} \left(\frac{1}{3^{4/3}}\right)^j = \frac{1 - \left(\frac{1}{3^{4/3}}\right)^{\log_3 n}}{1 - \frac{1}{3^{4/3}}}$

$$\approx \frac{1}{1 - \frac{1}{3^{4/3}}}$$

Hence :

$$T(n) = O(n^{4/3})$$

2. Solve following recurrence relation using Master's theorem:

(i) $T(n) = 3T(n/4) + n^2 \log n$

(ii) $T(n) = 9T(n/3) + n$

Solⁿ (i) $T(n) = 3T(n/4) + n \log n$

Comparing with $T(n) = aT(n/b) + f(n)$, we get

$$a=3 \quad b=4 \quad f(n) = n \log n$$

Solution: $T(n) = n^{\log_b a} [U(n)]$

$U(n)$ depends on $h(n)$

$$h(n) = \frac{f(n)}{n^{\log_b a}} = \frac{n \log n}{n^{\log_4 3}} = \frac{n \log n}{n} = \log n$$

$$\therefore \log_4 3 \approx 1$$

if $h(n) = (\log n)^i$

$$\therefore h(n) = (\log n)^i \quad \text{then} \quad U(n) = \frac{(\log_2 n)^{i+1}}{i+1}$$

$$U(n) = \frac{(\log_2 n)^2}{2}$$

$$U(n) = \log_2 n$$

Now, $T(n) = n^{\log_4 3} \log_2 n$

$$T(n) = O \log_2 n$$

(ii) $T(n) = 9T(n/3) + n$

$$T(n) = 9T(n/3) + n$$

Comparing with $T(n) = aT(n/b) + f(n)$
 $a = 9$ $b = 3$ $f(n) = n$

Solution: $T(n) = n \log_b^a [U(n)]$

$U(n)$ depends on $h(n)$

$$h(n) = \frac{f(n)}{n \log_b^a} = \frac{n}{n \log_3^9} = \frac{n}{n^2} = \frac{1}{n} = n^{-1}$$

$$U(n) = O(1)$$

Solⁿ: $T(n) = n \log_3^9 [O(1)]$

$$T(n) = O(n^2)$$

3 Solve using recursion tree:

(i) $T(n) = 4T(n/2) + n$

(ii) $T(n) = 16T(n/4) + n^2$

Ans

$$T(n) = 4T(n/2) + n \quad \text{--- (i)}$$

$$T(n/2) = 4T(n/4) + \frac{n}{2}$$

$$T(n/4) = 4T(n/8) + \frac{n}{4}$$

Put $T(n/2)$ in eqn (i)

$$T(n) = 4T[4T(n/4) + \frac{n}{2}] + n$$

$$T(n) = 16T(n/4) + 2n + n$$

$$T(n) = 16T(n/4) + 3n$$

Put $T(n/4)$ in above eqn:

$$T(n) = 16[4T(n/8) + \frac{n}{4}] + 3n$$

$$T(n) = 64T(n/8) + 4n + 3n$$

$$T(n) = 64T(n/8) + 7n$$

$$T(n) = 4^{k+1} T(n/2^{k+1}) + (2^{k+1} - 1)n \quad \text{--- (2)}$$

$$\text{let } n = 2^k \text{ so that } T(n/2^k) = T(1)$$

$$\therefore 4^k T(1) + (k^2 - 1)n$$

$$\frac{n}{2^k} = 1$$

$$n = 2^k$$

taking log

$$\log n = \log 2^k$$

$$\log n = k \log 2$$

$$\log n = k$$

Substitute $k = \log n$ in (2)

$$\begin{aligned} T(n) &= 4n^2 T(1/2) + (2n-1)n \\ &= n^2 \cdot c + n(2n-1)c \\ &= O(n^2) \end{aligned}$$

$$(ii) \quad T(n) = 16T(n/4) + n^2$$

$$T(n) = 16T(n/4) + n^2$$

$\therefore$ Here we can observe, that work done at each level is n^2 per.

$\therefore$ The problem is divided into 16 sub-problems at level, each of size $n/4$.

$$\text{Each problem has part of } (n/4)^2 = n^2/16$$

$$\therefore \text{ Since there has part of } (n/4)^2 = n^2/16$$

∴ Since there are 16 sub-problems
 $= 16 \times \frac{n^2}{16} = n^2$

Each sub-problem has size of $\frac{n}{4} = \frac{n}{16}$

Such that for each is $\left(\frac{n}{16}\right)^2 = \frac{n^2}{256}$

$$\frac{n^2}{4^k} = 1, \quad n = 4^k$$

$$k = \log_4 n$$

The Recursion tree has $\log_4 n$ levels

The constant is n^2

$$T(n) = n^2 (\log n)$$

$$T(n) = O(n^2 \log n)$$

Section - C

1. Find the maximum and minimum no from given list
 using D & C approach -

15, 89, 53, 78, 45, 6, 9, 36, 72, 19, 66, 49.

Ans

15	89	53	78	45	6	9	36	72	19	66	49
1	2	3	4	5	6	7	8	9	10	11	12

∴ Divide the list into half parts:

Take $i \rightarrow$ lower bound

$j \rightarrow$ upper bound

$$\text{mid} = \frac{i+j}{2} = \frac{6+12}{2} = 6.5 = 6$$

∴ So, Break the list into two parts having mid element in first list and rest in second list.

15 | 89 | 53 | 78 | 45 | 6 | ; | 9 | 36 | 72 | 19 | 66 | 49

15 | 89 | 53 | , | 78 | 45 | 6 | 9 | 36 | 72 | , | 19 | 66 | 49

15 | 89 | , | 53 | 78 | 45 | , | 6 | 9 | 36 | , | 72 | 19 | 66 | , | 49

min = 15 min = 3 min = 45 min = 6 min = 9 min = 19 min = 49
max = 89 max = 53 max = 78 max = 6 max = 36 max = 72 max = 66 max = 49

min = 15
max = 89

min = 6
max = 78

min = 9
max = 72

min = 19
max = 66

min = 6
max = 89

min = 9
max = 72

min = 6 (After comparing)
max = 89

Recursive Relation $T(n) = T(\log n) + c$

2. You are given two algo, Algo A & Algo B. Algorithm A has a time complexity of $O(n^2)$, while Algo B. has a time complexity of $O(n \log n)$. if you have to process an input of size 100, which algo would you choose, why?

Ans To determine which algorithm to choose for processing an input size $n = 1000$, let's analyze the time complexities.

1. Algorithm A : $O(n^2)$

• for $n = 1000$:

$$T_A(1000) = K \cdot (1000)^2 = 1000000K \text{ (for constant } K)$$

2. Algorithm B : $O(n \log n)$

$$T_A(1000) = K \cdot (1000) \log_2(1000)$$

$$\therefore \log_2 1000 \approx 10$$

$$T_B(1000) = K \cdot 1000 \cdot 10 = 10000K$$

$$\text{Comparison : } T_A(1000) \approx 1000000K$$

$$T_B(1000) \approx 10000K$$

Hence, Algo B with time complexity of $O(n \log n)$ is significantly more efficient than Algo A with $O(n^2)$ for an input size of 1000.

3. You are tasked with sorting a list of n integers using the Selection Sort and Insertion Sort algo. As part of the task, you need to perform a step-by-step analysis of the algo. on a given list of numbers.
Given list of integers: $[38, 29, 43, 3, 9, 82, 10]$.

Ans
Selection Sort: (select minimum element swap with first)
initial list: $[38, 29, 43, 3, 9, 82, 10]$

Pass 1: minimum element = 3

swap 3 with 38

list: $[3, 29, 43, 38, 9, 82, 10]$

Pass 2: Now minimum element from $[43, 38, 29, 82, 10] = 9$

swap 9 with 29

list: $[3, 9, 43, 38, 29, 82, 10]$

Pass 3: Now minimum element in $[43, 38, 29, 82, 10] = 10$

Swap 10 with 43
 list: [3 9 10 27 38 82 43]

Pass 5: Now minimum element from [38 82 43] = 38
 list remain same.

Pass 6: Now minimum element from [82 43] = 43
 swap 43 with 82

sorted list: [3 9 10 27 38 43 82]

• Insertion sort:

initial list: [38 27 43 3 9 82 10]

Pass 1: Take 27 and insert into sorted portion [38]

list: [27 38 43 3 9 82 10]

Pass 2: Take 43 (already in correct position)

list: [27 38 43 3 9 82 10]

Pass 3: Take 3 and insert into the sorted portion [27, 38, 43]

list: [3, 27, 38, 43, 9, 82, 10]

Pass 4: Take 9 and insert into sorted portion [3, 27, 38, 43]

list [3, 9, 27, 38, 43, 82, 10]

Pass 5: Take 82 and insert into sorted position [3, 9, 27, 38, 43]

list: [3, 9, 27, 38, 43, 82, 10]

Pass 6: Take 10 and insert into sorted portion [3, 9, 27, 38, 43, 82]

sorted list: [3, 9, 10, 27, 38, 43, 82]

Time Complexity:

Selection sort: $O(n^2)$ in all cases.

Insertion sort: $O(n^2)$ in worst case

$O(n)$ in best case (already sorted).